
ML-Based Modeling of GPU Power Requirements for Energy-Efficient GPU Recommendation

Avinash Gondela

University of California, San Diego
agondela@ucsd.edu

Bhavya Gupta

University of California, San Diego
b5gupta@ucsd.edu

Ksheer Sagar Agrawal

University of California, San Diego
ksagrawal@ucsd.edu

Saachi Shenoy

University of California, San Diego
svshenoy@ucsd.edu

Abstract

Modern GPUs are central to gaming, scientific computing, and artificial intelligence workloads. However, GPU selection is not only a performance decision; it also affects power consumption, thermal requirements, system cost, and infrastructure utilization. Existing GPU recommendations are often based on vendor specifications or static game requirements and may lead to over-provisioning, where users or schedulers select GPUs that are more powerful and more power-hungry than necessary.

This project proposes a machine learning based framework for energy-aware GPU recommendation. We use a GPU specification dataset from TechPowerUp to model GPU power requirements using board design TDP as the primary target and suggested PSU as a secondary system-level target. We use a PC video game requirements dataset to define workload-specific feasibility constraints such as VRAM, memory bandwidth, texture rate, pixel rate, and DirectX compatibility. Candidate GPUs are first filtered using these game-specific constraints and then ranked using performance-per-watt metrics. The final goal is to recommend the most energy-efficient feasible GPU rather than simply the most powerful or lowest-power GPU.

1 Problem Background and Motivation

Graphics Processing Units (GPUs) are widely used across domains such as gaming, machine learning, graphics rendering, scientific computing, and cloud infrastructure. As GPU capability has increased, power consumption has also become a major design and deployment concern. A high-performance GPU may provide strong compute capability, but it can also increase system power draw, cooling requirements, PSU provisioning needs, thermal pressure, and operational cost.

In gaming and consumer systems, users are often given minimum and recommended GPU requirements. However, these recommendations may not be optimized for energy efficiency. A user may select a GPU that is much more powerful than necessary, leading to unnecessary power consumption and higher cost. Similarly, in shared GPU clusters, assigning a high-power GPU to a relatively light workload may reduce GPU availability for workloads that actually require that level of performance.

Therefore, GPU recommendation should not only ask:

Which GPU is powerful enough?

It should instead ask:

Which GPU is powerful enough while using the least unnecessary power?

This project focuses on a static pre-deployment recommendation setting. We do not assume access to runtime GPU telemetry such as utilization, temperature, FPS, or live latency. Instead, we use publicly available static GPU specifications and game requirement data to estimate power and recommend efficient GPUs before deployment.

This setting is important because static game requirements are usually compatibility-oriented rather than efficiency-oriented. A minimum or recommended GPU requirement can indicate the approximate capability needed to run a game, but it does not tell the user which feasible GPU provides the best capability per watt or which feasible alternatives avoid unnecessary power demand. We therefore treat game requirements as feasibility constraints and use power-aware ranking only after compatibility has been enforced.

2 Problem Statement

Given a game or workload requirement and a list of candidate GPUs, our goal is to recommend a GPU that satisfies the workload’s minimum or recommended capability requirements while maximizing energy efficiency.

Formally, for a selected game q and a candidate GPU set G , we want to choose:

$$g^* = \arg \max_{g \in G_q} \frac{\text{PerformanceScore}(g)}{\widehat{\text{TDP}}(g)} \quad (1)$$

where G_q is the set of feasible GPUs for game q , and $\widehat{\text{TDP}}(g)$ is the predicted board power of GPU g .

The feasible set is defined as:

$$G_q = \{g \in G \mid C(g) \geq R(q)\} \quad (2)$$

where $C(g)$ represents the static capability vector of GPU g , and $R(q)$ represents the recommended requirement vector of game q .

In simple terms, our system first filters GPUs that are strong enough for the game and then ranks the remaining GPUs by performance-per-watt.

3 Datasets

We use two datasets.

3.1 TechPowerUp GPU Specification Dataset

The TechPowerUp GPU dataset contains specifications of real GPUs. This dataset acts as the candidate GPU database and is used to train power prediction models.

Important columns include:

3.2 PC Video Game Requirements Dataset

The game requirements dataset contains minimum and recommended system requirements for PC games. This dataset is used to define workload constraints.

Important columns include:

Column Group	Example Columns
GPU identity	Brand, Name, Graphics Processor_Architecture
Memory	Memory_Memory Size, Memory_Memory Type, Memory_Memory Bus, Memory_Bandwidth
Clock speeds	Clock Speeds_GPU Clock, Clock Speeds_Boost Clock, Clock Speeds_Memory Clock
Render configuration	Render Config_Shading Units, Render Config_TMUs, Render Config_ROPs, Render Config_Tensor Cores, Render Config_RT Cores
Theoretical performance	Theoretical Performance_FP32 (float), Theoretical Performance_Pixel Rate, Theoretical Performance_Texture Rate
Power targets	Board Design_TDP, Board Design_Suggested PSU
Compatibility	Graphics Features_DirectX, Graphics Features_OpenGL, Graphics Features_Vulkan

Table 1: Important columns from the TechPowerUp GPU specification dataset.

Column Group	Example Columns
Game identity	Name, Release_Date
Minimum GPU requirements	Min_GPU_Memory, Min_GPU_Memory_Bandwidth, Min_GPU_Texture_Rate, Min_GPU_Pixel_Rate, Min_GPU_PSU
Recommended GPU requirements	Recom_GPU_Memory, Recom_GPU_Memory_Bandwidth, Recom_GPU_Texture_Rate, Recom_GPU_Pixel_Rate, Recom_GPU_PSU
GPU units	Recom_GPU_TMUs, Recom_GPU_ROPs, Recom_GPU_Tensor_Cores
Compatibility	Recom_GPU_Direct_X, Recom_Direct_X
Memory requirements	Min_VRAM, Recom_VRAM, Min_RAM, Recom_RAM
External rating	Min_GPU_GD_RATING, Recom_GPU_GD_RATING

Table 2: Important columns from the PC video game requirements dataset.

4 Related Work

4.1 GPU Power and Performance Prediction

Prior work has shown that GPU power and performance can be modeled using hardware, workload, or runtime features. Hardware-level models often use detailed signals such as voltage, frequency, memory activity, cache behavior, instruction activity, and active compute units to estimate GPU power. Machine learning based approaches have also been used to predict GPU power or performance from measured workload behavior and hardware features Dutta et al., Ilager et al. [2020]

These methods motivate our use of learned power models for GPU recommendation. However, many prior approaches assume access to application execution, profiling traces, hardware counters, or runtime telemetry. Our setting is different. We target a static pre-deployment recommendation problem where the recommender must operate before the game or workload is run. Therefore, our model relies on static GPU specifications and game requirement constraints rather than runtime measurements.

4.2 Runtime Energy-Aware GPU Optimization

Another line of work reduces GPU energy consumption at runtime. These methods use signals such as utilization, temperature, current power draw, application progress, performance targets, and queue state to make control decisions. Examples include dynamic voltage and frequency scaling, workload-aware scheduling, and resource allocation policies that reduce energy while maintaining performance constraints Dutta et al., Ilager et al. [2020], throtLL’eM Authors [2024].

This line of work is closely related to our objective because it also treats GPU selection or control as a power-performance trade-off. However, runtime optimization assumes that the workload is already running or has been profiled. Our work targets an earlier decision point: choosing a GPU shortlist before deployment. Instead of changing frequency or scheduling at runtime, we use static GPU specifications and game requirements to identify feasible GPUs and rank them by energy-aware criteria.

4.3 Multi-Objective Recommendation and Learning-to-Rank

Our final problem is not only power prediction; it is recommendation. A useful GPU recommender should satisfy game constraints, preserve energy efficiency, and avoid repeatedly recommending the same small set of GPUs. This makes the problem naturally multi-objective. Multi-objective recommender systems study this setting, where relevance or suitability must be balanced against secondary objectives such as diversity, cost, or system constraints Jannach [2023].

We use this perspective to motivate our Pareto-aware learning-to-rank recommender. Pareto-efficient recommendation has been used to handle competing objectives without reducing the problem to a single unconstrained score Lin et al. [2019]. In our setting, a GPU is preferred when it is strong under power-efficiency criteria such as performance-per-watt, predicted TDP, predicted PSU, efficiency regret, and feature affinity. We then train a grouped learning-to-rank model, treating each game as a query and feasible GPUs as candidate items. This follows standard learning-to-rank practice, where ranking objectives such as NDCG are optimized within query groups XGBoost Developers [2024].

5 Our Positioning

Our project addresses a complementary problem to prior GPU power and performance modeling work. Instead of predicting runtime power for an already-running workload, we study static pre-deployment GPU recommendation. The goal is to use available GPU specifications and game requirement data to identify feasible GPUs before execution and rank them by energy-aware criteria.

Our project differs from prior work in three ways:

1. We operate before deployment rather than during runtime monitoring or control.
2. We use game requirements as compatibility constraints rather than as direct performance labels.
3. We focus on top- k recommendation, not only prediction. The final output is a ranked shortlist of energy-efficient feasible GPUs.

This scope is intentional. Our method does not claim to predict exact game-specific FPS, latency, or runtime energy. Instead, it builds a static-spec recommender using TDP and PSU as power proxies and hardware capability as a performance proxy. This makes the approach useful when runtime telemetry, FPS benchmarks, or measured energy traces are unavailable.

6 Methodology

6.1 Overview

7 Methodology

7.1 Overview

The project has four main components:

1. **Power prediction:** estimate GPU-side and system-level power proxies from static GPU specifications.
2. **Game requirement extraction:** convert each game requirement into a capability vector.
3. **Feasibility filtering:** construct a game-specific candidate set of GPUs that satisfy compatibility and capability constraints.
4. **Top- k recommendation:** rank feasible GPUs using power-aware and learning-to-rank methods.

The pipeline separates compatibility from ranking. Feasibility filtering prevents underpowered GPUs from being recommended, while the ranking stage compares feasible candidates using power, efficiency, diversity, and proxy relevance objectives.

Step	Dataset Used	Output
Prepare GPU data	TechPowerUp	Clean candidate GPU table
Train power model	TechPowerUp	Predicted TDP and optional predicted PSU
Build performance score	TechPowerUp	Static capability score for each GPU
Extract game requirements	Game dataset	Game-specific requirement vector
Filter GPUs	Both datasets	Feasible GPU set
Rank GPUs	Both datasets	Top- k efficient recommendations

Table 3: High-level project pipeline.

8 Power Prediction

8.1 Why TDP is the Primary Target

We use Board Design__TDP as the primary prediction target because it is closer to the actual GPU board power. TDP is not exact runtime power, but it is GPU-specific and more directly related to energy efficiency.

We use Board Design__Suggested PSU as a secondary target because PSU is a system-level provisioning recommendation. Suggested PSU includes not only the GPU but also CPU, motherboard, storage, fans, and safety margin.

Therefore:

Target	Meaning	Role in Project
Board Design__TDP	Approximate GPU board power	Primary power target
Board Design__Suggested PSU	Recommended full-system PSU	Secondary provisioning target

Table 4: Power targets used in the project.

This design protects against the criticism that PSU is not exact GPU power. We explicitly treat PSU as a secondary system-level metric.

8.2 Power Model Input Features

The power model learns:

$$\text{GPU Specs} \rightarrow \widehat{\text{TDP}} \quad (3)$$

Candidate input features include:

- Memory size.
- Memory type.
- Memory bus width.
- Memory bandwidth.
- GPU clock.
- Boost clock.
- Memory clock.
- Shading units or cores.
- TMUs.
- ROPs.
- Tensor cores.
- RT cores.
- FP32 theoretical performance.
- Texture rate.
- Pixel rate.
- Architecture.
- Process size.

8.3 Power Models

We evaluate the following regression models:

Model	Reason for Use
Linear Regression	Simple interpretable baseline
Ridge Regression	Handles correlated features better than ordinary linear regression
Lasso Regression	Helps identify important features through sparsity
Random Forest Regressor	Captures nonlinear feature interactions
XGBoost or LightGBM	Strong tabular models; expected to perform well
Small Neural Network	Optional model if dataset size is sufficient

Table 5: Candidate models for TDP and PSU prediction.

9 Performance Modeling

9.1 Why Performance Modeling is Needed

Choosing the lowest-power GPU is not enough. A low-power GPU may be too weak for the selected game. Therefore, we need a performance proxy to ensure that a GPU is strong enough and to rank feasible GPUs by energy efficiency.

The important distinction is:

Power tells us how much energy the GPU may consume. Performance tells us how capable the GPU is. Efficiency is performance divided by power.

9.2 Game Requirements as Thresholds

The game dataset does not necessarily recommend a GPU by name. Instead, it provides minimum and recommended GPU specification columns. These columns are treated as a requirement vector.

For a game g , the recommended requirement vector is:

$$R(q) = [\text{VRAM}, \text{Bandwidth}, \text{TextureRate}, \text{PixelRate}, \text{TMUs}, \text{ROPs}, \text{DirectX}] \quad (4)$$

This vector defines the approximate capability threshold required for the game.

9.3 Candidate GPU Capability Vector

Each TechPowerUp GPU has a capability vector:

$$C(g) = [\text{VRAM}, \text{Bandwidth}, \text{TextureRate}, \text{PixelRate}, \text{TMUs}, \text{ROPs}, \text{DirectX}] \quad (5)$$

A GPU is considered feasible if it satisfies the game requirement vector.

10 Feasibility Filtering

Feasibility filtering is applied before ranking. This prevents weak GPUs from being recommended simply because they have low power.

The basic feasibility constraints are:

$$\text{VRAM}(g) \geq \text{VRAM}_{req}(q) \quad (6)$$

$$\text{DirectX}(g) \geq \text{DirectX}_{req}(q) \quad (7)$$

$$\text{Bandwidth}(g) \geq \alpha \cdot \text{Bandwidth}_{req}(q) \quad (8)$$

$$\text{TextureRate}(g) \geq \alpha \cdot \text{TextureRate}_{req}(q) \quad (9)$$

$$\text{PixelRate}(g) \geq \alpha \cdot \text{PixelRate}_{req}(q) \quad (10)$$

where α is a tolerance factor, such as 0.9. The tolerance is useful because strict column-wise filtering may incorrectly reject newer GPUs that have better architecture but slightly lower values in one raw feature.

10.1 Hard vs Soft Constraints

Not all columns should be used as hard constraints. Some are better used in the performance score.

Feature	Constraint Type	Reason
VRAM	Hard	Insufficient VRAM can prevent smooth execution
DirectX	Hard	Compatibility requirement
Memory bandwidth	Soft or tolerant hard	Important but architecture-dependent
Texture rate	Soft or tolerant hard	Important for rendering but not sufficient alone
Pixel rate	Soft or tolerant hard	Important for output/rendering
TMUs and ROPs	Soft	Useful but architecture-dependent
Clock speed	Soft	Not comparable across architectures
Memory type	Soft/categorical	Useful but not directly ordered in all cases

Table 6: Suggested use of common features in feasibility filtering.

11 Performance Score

11.1 Transparent Hardware Performance Score

Since the datasets do not contain true game-specific FPS, we define a transparent static hardware capability score. This score does not claim to predict real FPS. It approximates GPU capability using static hardware features.

Let each feature be normalized across candidate GPUs using min-max normalization:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (11)$$

A possible performance score is:

$$\text{PerfScore}(g) = 0.30 \cdot \text{FP32}'(g) + 0.25 \cdot \text{Bandwidth}'(g) + 0.20 \cdot \text{TextureRate}'(g) + 0.20 \cdot \text{PixelRate}'(g) + 0.05 \cdot \text{VRAM}'(g) \quad (12)$$

This score is interpretable because each term corresponds to a known GPU capability.

11.2 GD Rating as a Secondary Learned Proxy

The game dataset contains `Recom.GPU_GD_RATING`. This can be used as a secondary performance proxy. However, GD rating is likely a black-box external score, so it should not be the only performance metric.

We can train a model:

$$\text{Shared GPU Features} \rightarrow \widehat{\text{GD Rating}} \quad (13)$$

using only columns that are mappable between the game dataset and the TechPowerUp dataset.

Shared features may include:

- VRAM.
- Memory bandwidth.
- Boost clock.
- TMUs.
- ROPs.
- Texture rate.
- Pixel rate.

The predicted GD rating can then be applied to TechPowerUp GPUs and used in a secondary ranking method:

$$\text{Efficiency}_{GD}(g) = \frac{\widehat{\text{GD Rating}}(g)}{\widehat{\text{TDP}}(g)} \quad (14)$$

11.3 Why Use Both Scores

The transparent performance score is reproducible and explainable. The GD-rating model may capture patterns from an external rating system but is less transparent. Therefore, we evaluate both.

Performance Metric	Strength	Weakness
Transparent hardware score	Interpretable and reproducible	Not true game FPS
Predicted GD rating	Learned from external rating data	Black-box target and may encode hidden assumptions

Table 7: Comparison of performance proxy choices.

12 Recommendation Objective

After filtering feasible GPUs, we rank them by energy efficiency.

The main efficiency score is:

$$\text{Efficiency}(g) = \frac{\text{PerfScore}(g)}{\widehat{\text{TDP}}(g)} \quad (15)$$

The final recommendation is:

$$g^* = \arg \max_{g \in G_q} \text{Efficiency}(g) \quad (16)$$

where G_q is the feasible GPU set for game q .

We also evaluate an alternate ranking using predicted GD rating:

$$g_{GD}^* = \arg \max_{g \in G_q} \frac{\widehat{\text{GD Rating}}(g)}{\widehat{\text{TDP}}(g)} \quad (17)$$

13 Working Example

Suppose a game has the following recommended requirements:

Requirement	Value
Recommended VRAM	6 GB
Recommended memory bandwidth	300 GB/s
Recommended texture rate	200 GTexel/s
Recommended pixel rate	80 GPixel/s
Recommended DirectX	12

Table 8: Example game requirement vector.

Candidate GPUs are evaluated as follows:

GPU	PerfScore	Predicted TDP	Perf/Watt	Feasible?
GTX 1060	70	120W	0.58	No
RTX 2060	85	160W	0.53	Yes
RTX 3060	90	170W	0.53	Yes
RTX 4060	88	115W	0.77	Yes
RTX 4070	110	200W	0.55	Yes

Table 9: Example recommendation calculation.

Although RTX 4070 has the highest raw performance score, RTX 4060 has the best performance-per-watt among feasible GPUs. Therefore, the recommender selects RTX 4060.

14 Baselines

Baselines are simple methods that help test whether our proposed method is actually useful. Without baselines, the project cannot show whether ML and performance-per-watt ranking improve over simple rules.

We compare against the following baselines:

Baseline	Description
Lowest TDP feasible	Filter feasible GPUs and choose the one with lowest known or predicted TDP
Lowest PSU feasible	Filter feasible GPUs and choose the one with lowest suggested PSU
Highest performance feasible	Filter feasible GPUs and choose the GPU with highest performance score
Random feasible GPU	Randomly select one feasible GPU
Rule-based score	Use manually defined thresholds and weights over VRAM, bandwidth, texture rate, and pixel rate
Transparent Perf/TDP	Rank feasible GPUs by transparent hardware performance score divided by predicted TDP
Predicted GD/TDP	Rank feasible GPUs by predicted GD rating divided by predicted TDP

Table 10: Baselines and proposed ranking methods.

15 Evaluation Plan

Evaluation is divided into two parts: prediction evaluation and recommendation evaluation.

15.1 Prediction Evaluation

For TDP and PSU prediction, we evaluate regression performance using:

- Mean Absolute Error (MAE).
- Root Mean Squared Error (RMSE).
- R^2 score.
- Inference latency.

For example:

Model	MAE ↓	RMSE ↓	R^2 ↑
Linear Regression	—	—	—
Ridge Regression	—	—	—
Random Forest	—	—	—
XGBoost/LightGBM	—	—	—
Small Neural Network	—	—	—

Table 11: Template for TDP prediction evaluation.

15.2 Recommendation Evaluation

Since all recommendation methods use the same feasibility filter, feasibility rate will usually be 100% and constraint violation rate will usually be 0%. Therefore, the more important metrics are efficiency and over-provisioning metrics.

15.2.1 Average Selected TDP

This measures the average power requirement of the GPUs selected by a method:

$$\text{AvgSelectedTDP} = \frac{1}{N} \sum_{i=1}^N \widehat{\text{TDP}}(g_i) \quad (18)$$

Lower is better, assuming feasibility is maintained.

15.2.2 Average Selected PSU

This measures average system-level PSU requirement:

$$\text{AvgSelectedPSU} = \frac{1}{N} \sum_{i=1}^N \widehat{\text{PSU}}(g_i) \quad (19)$$

Lower values indicate lower system provisioning requirements.

15.2.3 Average Performance Score

This measures the average capability of selected GPUs:

$$\text{AvgPerf} = \frac{1}{N} \sum_{i=1}^N \text{PerfScore}(g_i) \quad (20)$$

Higher is not always better, because excessive performance may indicate over-provisioning.

15.2.4 Average Performance-per-Watt

This is the primary efficiency metric:

$$\text{AvgPerfPerWatt} = \frac{1}{N} \sum_{i=1}^N \frac{\text{PerfScore}(g_i)}{\widehat{\text{TDP}}(g_i)} \quad (21)$$

Higher is better.

15.2.5 Performance Over-Provisioning

Performance over-provisioning measures how much extra performance the selected GPU provides beyond the game requirement.

$$\text{PerfOverprov}(q, g) = \frac{\text{PerfScore}(g) - \text{PerfScore}_{req}(q)}{\text{PerfScore}_{req}(q)} \quad (22)$$

Lower is better, as long as the GPU remains feasible.

For example, if a game requires a performance score of 80 and the selected GPU has a score of 100:

$$\frac{100 - 80}{80} = 0.25 \quad (23)$$

This means the GPU is 25% over-provisioned in performance.

15.2.6 Power Over-Provisioning

Power over-provisioning measures how much more power the selected GPU uses compared to the lowest-power feasible GPU for that game.

$$\text{PowerOverprov}(q, g) = \frac{\widehat{\text{TDP}}(g) - \min_{h \in G_q} \widehat{\text{TDP}}(h)}{\min_{h \in G_q} \widehat{\text{TDP}}(h)} \quad (24)$$

Lower is better.

15.2.7 Efficiency Regret

Efficiency regret measures how far the selected GPU is from the best possible efficiency among feasible GPUs.

$$\text{EfficiencyRegret}(q, g) = \frac{\max_{h \in G_q} \text{Efficiency}(h) - \text{Efficiency}(g)}{\max_{h \in G_q} \text{Efficiency}(h)} \quad (25)$$

Lower is better.

15.3 Recommendation Comparison Table

The final recommendation comparison can be reported as:

Method	Avg TDP ↓	Avg PSU ↓	Perf/Watt ↑	Perf Overprov ↓	Power Overprov ↓
Lowest TDP feasible	—	—	—	—	—
Lowest PSU feasible	—	—	—	—	—
Highest performance feasible	—	—	—	—	—
Random feasible	—	—	—	—	—
Rule-based score	—	—	—	—	—
Transparent Perf/TDP	—	—	—	—	—
Predicted GD/TDP	—	—	—	—	—

Table 12: Template for comparing recommendation methods.

16 Expected Outcomes

We expect the project to show that:

1. GPU TDP and suggested PSU can be reasonably predicted from static GPU specifications.
2. TDP is a better primary power target than suggested PSU because it is more GPU-specific.
3. Game recommended specifications can be used as approximate feasibility thresholds.
4. Performance-per-watt ranking can reduce unnecessary power usage compared to highest-performance selection.
5. Learned or transparent performance proxies can provide better recommendation behavior than naive lowest-power selection.

17 Limitations

This project has several limitations.

First, TDP is not exact runtime power. Actual GPU power depends on workload behavior, utilization, temperature, voltage, frequency, and runtime scheduling.

Second, suggested PSU is a system-level recommendation and includes non-GPU components and safety margins. Therefore, it should not be treated as exact GPU power.

Third, the performance score is a static hardware capability proxy. It does not predict true game-specific FPS, latency, or user experience.

Fourth, the game dataset provides static minimum and recommended specifications. It does not provide measured FPS, resolution, graphics settings, driver version, or runtime telemetry.

Fifth, strict column-wise filtering can reject newer GPUs that are architecturally more efficient but have lower values in one raw feature. We address this using tolerance factors and performance scoring, but the limitation remains.

Finally, the project does not solve full cloud gaming GPU placement because it does not include network latency, region, queue time, GPU availability, or live utilization.

18 Future Work

Future extensions include:

- Incorporating real FPS benchmarks for game-specific performance modeling.
- Adding cloud deployment constraints such as GPU availability, region, cost, and network latency.
- Using runtime telemetry such as utilization, temperature, and memory bandwidth for workload-specific power prediction.
- Extending the recommender to ML training and inference workloads.
- Adding cost-aware metrics such as dollars per frame or dollars per token.
- Building an LLM interface for natural language queries and explanation.

19 Conclusion

This project proposes a static-spec energy-aware GPU recommendation framework. The core idea is to train GPU power models using TechPowerUp specifications, use game requirement data as feasibility constraints, and recommend the most efficient feasible GPU using performance-per-watt ranking.

The project is intentionally scoped as a pre-deployment recommender. It does not claim to predict exact FPS, runtime power, or cloud gaming latency. Instead, it provides a practical and interpretable method for reducing power over-provisioning in GPU selection.

20 Top- k GPU Recommendation Evaluation

20.1 Evaluation Setup

We evaluate the proposed system as a top- k energy-aware GPU recommender. Given a game requirement vector and a database of candidate GPUs, the goal is to return a shortlist of feasible GPUs that satisfy compatibility constraints while reducing unnecessary power demand. For each game q , we first construct a hard-feasible GPU set G_q . The feasibility filter uses compatibility and capability constraints including VRAM capacity, DirectX support, memory bandwidth, texture rate, and pixel rate. This ensures that all ranking methods operate on candidates that satisfy the same game-level requirements.

The evaluation uses the recommended requirement track. All methods use the same game set, the same hard-feasible candidate sets, and the same predicted or available TDP and PSU values. Since direct game-specific FPS measurements and runtime power traces are unavailable, we evaluate recommendation quality using specification-derived proxies: predicted TDP and PSU for power demand, performance-per-watt for efficiency, efficiency regret for distance from the best feasible efficiency option, and diversity metrics to measure recommendation collapse.

20.2 Compared Methods

We compare six top- k recommendation strategies. These methods are designed to separate feature-affinity retrieval, direct efficiency optimization, utility-based ranking, and learned ranking.

- **KNN50-Feasible**: retrieves the 50 nearest feasible GPUs using feature affinity between the game requirement vector and GPU capability vector.
- **KNN50-Feasible-PPW-Top5**: reranks the feasible KNN50 candidate set by performance-per-watt and returns the top 5 GPUs.
- **Power-Top5**: directly selects the top 5 feasible GPUs ranked by performance-per-watt.
- **UtilityFormula-Top5**: selects the top 5 feasible GPUs using a hand-designed multi-objective utility score.
- **ML-Utility-Top5**: trains a pointwise regression model to predict the hand-designed utility score and returns the top 5 GPUs by predicted utility.
- **LTR-Utility-Top5**: trains a Pareto-aware learning-to-rank model using proxy relevance labels and returns the top 5 GPUs by predicted ranking score.

The performance-per-watt score is defined as:

$$\text{PPW}(g) = \frac{\text{PerfScore}(g)}{\widehat{\text{TDP}}(g)}, \quad (26)$$

where $\text{PerfScore}(g)$ is the static GPU capability score and $\widehat{\text{TDP}}(g)$ is the predicted or available board power estimate.

The direct power-aware recommender is:

$$\text{PowerTop5}(q) = \text{Top5}_{g \in G_q} \frac{\text{PerfScore}(g)}{\widehat{\text{TDP}}(g)}. \quad (27)$$

This baseline is intentionally strong because it directly optimizes the primary efficiency metric. However, as shown below, optimizing only this ratio can collapse recommendations to a small set of globally efficient GPUs.

20.3 Utility and Learning-to-Rank Objectives

The utility-based methods extend pure performance-per-watt ranking by adding secondary terms for power demand, PSU requirement, feasibility margin, and right-sizing. The utility score is:

$$S(q, g) = 0.50S_{\text{PPW}}(q, g) + 0.15S_{\text{TDP}}(q, g) + 0.10S_{\text{PSU}}(q, g) + 0.15S_{\text{margin}}(q, g) + 0.10S_{\text{right-size}}(q, g). \quad (28)$$

Here, S_{PPW} is the normalized performance-per-watt score, S_{TDP} and S_{PSU} reward lower predicted TDP and PSU, S_{margin} rewards feasible performance margin, and $S_{\text{right-size}}$ penalizes excessive over-provisioning. Power efficiency receives the largest weight because the primary objective is energy-aware recommendation.

UtilityFormula-Top5 directly ranks feasible GPUs by $S(q, g)$. ML-Utility-Top5 trains a regression model to predict this same proxy utility score from game-GPU pair features. This model is included as a diagnostic baseline: if it produces nearly the same ranking as UtilityFormula-Top5, then the learned model is mostly reproducing the hand-designed objective rather than discovering a distinct recommendation policy.

To better match the structure of top- k recommendation, we also evaluate LTR-Utility-Top5. This method is motivated by multi-objective recommendation and ranking, where recommendation quality must balance multiple objectives rather than optimize a single scalar score Jannach [2023], Momma et al. [2020]. We use a Pareto-aware labeling strategy inspired by Pareto-efficient recommendation, where candidates are preferred if they are not dominated across competing objectives Lin et al. [2019].

LTR-Utility-Top5 treats each game as a query and feasible GPUs as candidate items. Since ground-truth recommendation labels are unavailable, we construct proxy relevance labels from power-efficiency-first criteria. Candidate GPUs are assigned relevance labels from 0 to 3 using PPW, predicted TDP, predicted PSU, efficiency regret, feature-affinity distance, and Pareto-front membership. Higher labels correspond to candidates that are more power-efficient, less power-demanding, and closer to the Pareto-efficient region of the feasible set.

Table 13: Top- k recommendation results on the recommended requirement track. TDP, PSU, and efficiency regret are lower-is-better. PPW, unique GPUs, NDCG@5, and Recall@5 are higher-is-better. Top-1 share is lower-is-better because lower concentration indicates less recommendation collapse.

Method	TDP ↓	PSU ↓	PPW ↑	EffRegret ↓	Unique ↑	Top1Share ↓	NDCG@5 ↑	Recall@5 ↑
KNN50-Feasible	141.40	383.69	0.000647	0.879	977	0.137	0.592	0.160
KNN50-Feasible-PPW-Top5	78.67	267.80	0.001174	0.779	231	0.175	0.989	1.000
Power-Top5	56.70	231.46	0.004113	0.258	19	0.938	0.993	1.000
UtilityFormula-Top5	56.06	230.07	0.004085	0.263	32	0.938	0.993	1.000
ML-Utility-Top5	66.65	248.46	0.004005	0.277	38	0.938	0.988	1.000
LTR-Utility-Top5	50.33	221.80	0.003658	0.340	56	0.351	1.000	1.000

The ranker is trained to order feasible GPUs within each game-specific candidate group using a LambdaMART-style learning-to-rank objective implemented through XGBoost XGBoost Developers [2024]. At inference time, the model scores all feasible GPUs for a test game and returns the top 5 GPUs. This formulation better matches the recommendation problem than pointwise utility regression because it learns relative ordering within each game-specific candidate set.

20.4 Evaluation Metrics

We evaluate each method using power, efficiency, diversity, and ranking metrics:

- **Average TDP:** average predicted GPU-side board power of the selected top- k GPUs.
- **Average PSU:** average predicted system-level PSU requirement.
- **Average PPW:** average performance-per-watt of the selected GPUs.
- **Efficiency regret:** gap from the best feasible PPW option for the same game.
- **Unique GPUs:** number of distinct GPUs selected across all games.
- **Top-1 share:** fraction of games for which the most frequent GPU appears as the top recommendation.
- **NDCG@5:** ranking quality under the proxy relevance labels.
- **Recall@5:** fraction of highest-relevance proxy candidates recovered in the top 5.

Over-provisioning is treated only as a proxy because game performance scores and GPU performance scores are normalized separately. Therefore, it should not be interpreted as a direct measurement of true runtime performance surplus. Similarly, NDCG@5 and Recall@5 evaluate consistency with the constructed proxy relevance labels, not real user choices or measured FPS outcomes.

20.5 Aggregate Results

Table 13 reports the aggregate results. KNN50-Feasible provides the highest catalog diversity, but it performs poorly on power-efficiency metrics. Reranking the KNN candidate set by PPW improves efficiency, but remains far below direct power-aware ranking. Power-Top5 achieves the highest PPW and lowest efficiency regret, which is expected because it directly optimizes performance-per-watt. However, it selects only 19 unique GPUs and has a high top-1 share, indicating recommendation collapse.

UtilityFormula-Top5 and ML-Utility-Top5 remain close to Power-Top5, showing that the pointwise ML utility model mostly learns the hand-designed utility objective. In contrast, LTR-Utility-Top5 provides a stronger recommender-style trade-off. It achieves the lowest average TDP and PSU, increases the number of unique selected GPUs to 56, and reduces top-1 share from 0.938 to 0.351 while maintaining substantially higher PPW than the KNN-based methods.

Figure 1 visualizes the same trends. Power-Top5 is strongest on pure PPW and efficiency regret, while LTR-Utility-Top5 provides the strongest balance across power demand, recommendation diversity, and proxy ranking quality.

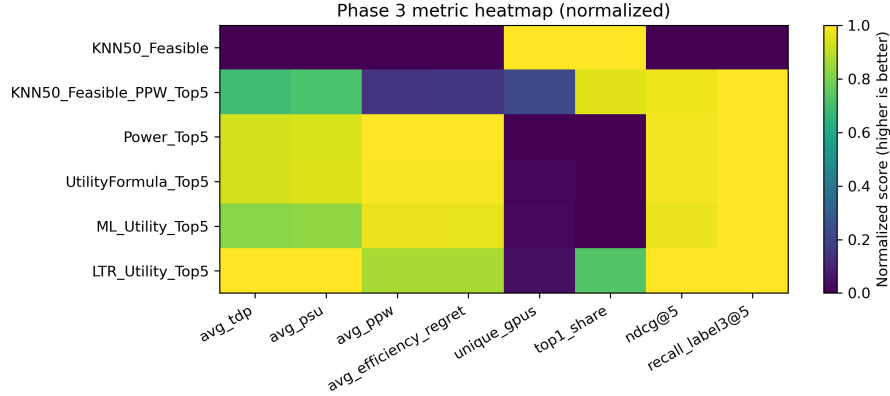


Figure 1: Aggregate comparison of top- k GPU recommendation methods. Values are normalized so that higher is better. Power-Top5 is strongest on pure PPW and efficiency regret, while LTR-Utility-Top5 provides the strongest balance across power, diversity, and proxy ranking quality.

Table 14: Diversity-efficiency comparison. LTR-Utility-Top5 sacrifices a small amount of PPW relative to Power-Top5, but substantially improves recommendation diversity and reduces top recommendation concentration.

Method	PPW $\uparrow$	Unique GPUs $\uparrow$	Top1Share $\downarrow$
KNN50-Feasible	0.000647	977	0.137
KNN50-Feasible-PPW-Top5	0.001174	231	0.175
Power-Top5	0.004113	19	0.938
UtilityFormula-Top5	0.004085	32	0.938
ML-Utility-Top5	0.004005	38	0.938
LTR-Utility-Top5	0.003658	56	0.351

20.6 Recommendation Diversity and Collapse

Table 14 isolates the diversity-efficiency trade-off. KNN retrieval has high diversity but poor efficiency. Direct PPW ranking has excellent efficiency but very low diversity. LTR-Utility-Top5 occupies a more useful middle ground: it keeps PPW much higher than KNN-based retrieval while substantially reducing the concentration observed in Power-Top5.

20.7 Method Overlap Analysis

Figure 2 compares overlap between top-5 recommendation sets. Power-Top5, UtilityFormula-Top5, and ML-Utility-Top5 have high overlap, showing that the pointwise utility regressor largely reproduces the hand-designed utility behavior. LTR-Utility-Top5 has only moderate overlap with these methods. This indicates that it preserves some efficient candidates but also introduces different GPUs into the shortlist.

This result is important because it shows that learning-to-rank changes the recommendation behavior. Unlike ML-Utility-Top5, which learns to approximate a formula, LTR-Utility-Top5 learns a grouped ranking objective and produces a more diverse top- k list.

20.8 Result Interpretation

The evaluation shows a clear progression across recommendation formulations. KNN-based retrieval captures feature affinity but is weak on power efficiency. Direct Power-Top5 ranking strongly optimizes performance-per-watt, but collapses to a small set of repeated GPUs. The pointwise ML utility regressor learns the hand-designed utility score accurately, but produces recommendations very similar to the utility formula.

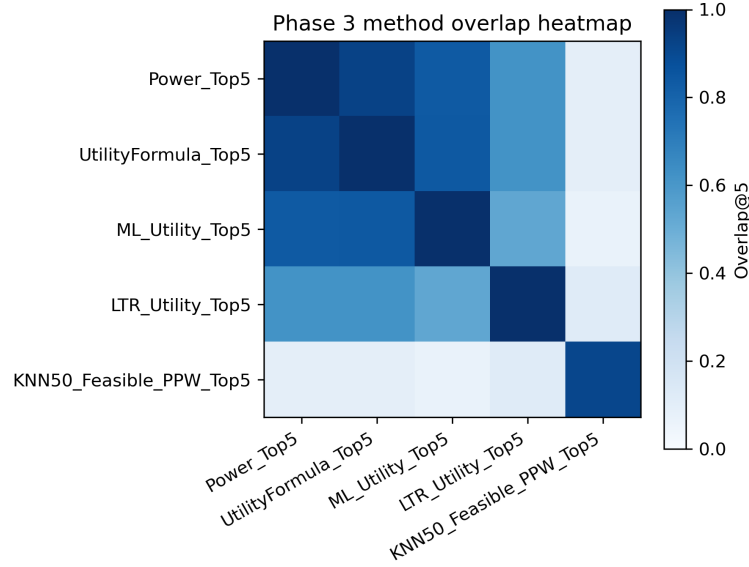


Figure 2: Average overlap@5 between recommendation methods. LTR-Utility-Top5 partially overlaps with power-aware methods but does not simply copy them, indicating a distinct ranking behavior.

The Pareto-aware learning-to-rank model provides the strongest overall recommender behavior. It does not beat Power-Top5 on pure PPW, which is expected because Power-Top5 directly optimizes that metric. However, it reduces average TDP and PSU, improves diversity, lowers the concentration of the top recommendation, and achieves strong proxy ranking metrics. This makes LTR-Utility-Top5 the most balanced method under the current specification-derived evaluation setting.

The main limitation is that relevance labels are proxy labels derived from power-efficiency-first criteria, not real user choices, measured FPS, or runtime energy traces. Therefore, the results should be interpreted as evidence that learning-to-rank can improve specification-derived energy-aware recommendation, not as proof of true game-level runtime optimality.

References

- Bishwajit Dutta, Vignesh Adhinarayan, and Wu-chun Feng. Gpu power prediction via ensemble machine learning for dvfs space exploration. <https://vtechworks.lib.vt.edu/server/api/core/bitstreams/561a97e2-cf36-40e5-a956-299ad6245458/>
- Shashikant Ilager et al. A data-driven frequency scaling approach for deadline-aware energy efficient execution on gpus. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing*, 2020.
- Dietmar Jannach. A survey on multi-objective recommender systems. *Frontiers in Big Data*, 2023.
- Xiao Lin et al. A pareto-efficient algorithm for multiple objective optimization in e-commerce recommendation. In *Proceedings of the 13th ACM Conference on Recommender Systems*, 2019.
- Michinari Momma, Alireza Bagheri Garakani, Nanxun Ma, and Yi Sun. Multi-objective ranking via constrained optimization. In *Proceedings of The Web Conference*, 2020.
- throttLL’eM Authors. Slo-aware gpu frequency scaling for energy efficient llm inference. <https://arxiv.org/html/2408.05235v1>, 2024.
- XGBoost Developers. Learning to rank. https://xgboost.readthedocs.io/en/stable/tutorials/learning_to_rank.html